SISTEMA ESTRATÉGICO COMUNAL

Guía técnica de integración municipal

Base municipal S.E.C. · Aplicación web y servidor

Instalación local, primer acceso, conexión de servicios, diagnóstico y puesta en producción.

Edición 1.1 · 13 de noviembre de 2026

Para equipos informáticos municipales, responsables de operación e integradores.

ASOCIACIÓN ESTRATÉGICA COMUNAL

Consultar al asistente de integración S.E.C.

Acceso desde el navegador o mediante el código QR.

01. Alcance y ruta de implementación

S.E.C. es una base web y API para coordinación de seguridad pública municipal. Incluye acceso por perfiles, paneles, mapas, flujos de trabajo y puntos de conexión. Cada municipio prepara su propia infraestructura, configura los servicios y aprueba las pruebas de aceptación antes de operar.

Elija la prueba según su objetivo

1. **Vista previa**. Permite recorrer las pantallas con usuarios ficticios. La guía de la sección 03 indica cómo iniciarla.
2. **Acceso real aislado**. La web envía el formulario a la API y el servidor autentica contra una base nueva. Compruebe inicio y cierre de sesión según las secciones 04 y 05.
3. **Piloto funcional**. Conecte datos operativos y los servicios que requiere el municipio. Pruebe cada flujo según las secciones 06 a 10.
4. **Producción**. Publique solo la compilación aprobada y ejecute las verificaciones de la sección 11.

Qué recibe el municipio

El paquete incluye código web React, API Python, scripts de prueba y documentación de integración. Los paneles cubren administración, dirección, operación, patrullaje, ciudadanía, fiscalía y patentes. La app móvil nativa y el módulo V.I.F. están fuera del alcance de esta base.

La copia parte sin base de datos entregada, registros operativos, cuentas reales, credenciales, adjuntos ni identidad institucional. MongoDB es el motor compatible con la autenticación existente. Cada municipio prepara una instancia propia si decide conservar ese motor.

Cómo circula la información

Navegador, API municipal y base de datos. El navegador se comunica con la API, que comprueba identidad y permisos antes de consultar la base. Los conectores se ocupan de los datos operativos, archivos, cartografía y servicios opcionales. La interfaz se organiza según el rol que devuelve el servidor.

Índice de trabajo

02 Preparación municipal · 03 Vista previa · 04 API y primer administrador · 05 Web y sesión · 06 Usuarios y permisos · 07 Datos y archivos · 08 Mapas y cámaras · 09 Planillas Microsoft · 10 Comunicaciones y alertas · 11 Producción y continuidad · 12 Diagnóstico rápido · 13 Consulta técnica y referencias.

02. Preparación municipal

Ficha de levantamiento

- **Responsables.** Identifique a la contraparte técnica, a quien aprueba los roles, al responsable de seguridad y a la unidad que recibe incidentes de operación.
- **Infraestructura.** Registre proveedor o centro de datos, dominios de prueba y producción, ubicación de la API, método de despliegue, monitoreo y responsable de renovación de certificados.
- **Identidad.** Defina el método de acceso, el correo institucional del primer administrador, las redes autorizadas para operadores y el procedimiento de recuperación.
- **Datos y archivos.** Indique motor, modelo, volúmenes, frecuencia de respaldo, retención y responsable de probar la restauración.
- **Operación territorial.** Indique área de cobertura, centro y zoom inicial, fuentes cartográficas, inventario de puntos y perfiles habilitados.
- **Servicios opcionales.** Indique si usará Microsoft, correo, Web Push, SMS, avisos móviles o videollamadas. Identifique al responsable y el flujo que espera habilitar.
- **Identidad visual.** Entregue nombre institucional, recursos gráficos aprobados, teléfonos, enlaces y textos públicos aplicables.

Antes de ejecutar comandos

Obtenga acceso autorizado al repositorio privado y registre la versión o el commit de la copia que va a instalar. Abra una terminal en la carpeta que contiene `main.py`, `requirements.txt`, `scripts` y `panic-app`. Compruebe que dispone de Python, Node.js y npm según el entorno validado por el integrador. Los ejemplos del manual usan PowerShell en Windows.

Separe los entornos de prueba y producción desde el comienzo. Asigne a cada uno base, almacenamiento, credenciales y respaldos propios. Para ensayar flujos, prepare usuarios ficticios y datos sintéticos que no se confundan con alertas reales.

Dónde va cada configuración

`.env.example` enumera las variables del **servidor**. Configure sus valores en el entorno de ejecución o gestor de secretos. Uvicorn no carga el archivo de ejemplo por defecto.

`panic-app/.env.example` enumera la configuración **pública de la web**. En desarrollo puede usar `panic-app/.env.local`. Las variables `REACT_APP_*` se incorporan al código entregado al navegador. Reserve las credenciales, claves de sesión y cadenas privadas de conexión para el servidor.

03. Vista previa local

Use este modo para revisar navegación, textos y presentación. El script activa el acceso ficticio de desarrollo y dirige la web al puerto 3014.

Iniciar

```
Set-Location '<ruta de SEC-COPIA-BASE>'
Set-Location .\panic-app
npm.cmd ci
Set-Location ..
powershell -ExecutionPolicy Bypass -File .\scripts\start-login-preview.ps1
```

Abra `http://127.0.0.1:3014/`. La lista de perfiles de muestra está en `panic-app/src/localPreviewAuth.js`. El script local configura la contraseña de demostración. Use esas cuentas únicamente en esta vista de desarrollo.

Qué comprobar

- Abra el panel de uno o más perfiles y recorra los módulos que integrará el municipio.
- Los mapas y paneles pueden mostrar una estructura vacía. En esta etapa se valida navegación.
- Cierre la sesión y pruebe otro perfil. Para pasar al acceso real, detenga esta vista con Ctrl+C en su terminal y siga la sección 04.

Si no inicia

Si PowerShell bloquea `npm.ps1`, ejecute `npm.cmd ci` desde `panic-app`. Si el puerto 3014 está ocupado, identifique qué proceso lo usa y cierre la vista anterior desde su terminal antes de iniciar otra. Si persiste un error de compilación, anote el primer error de la terminal y compruebe la versión de Node y el directorio actual.

04. API y primer administrador

Terminal 1 · Base de autenticación

Para conservar el conector existente, arranque MongoDB en una instancia de desarrollo propia y enlazada a `127.0.0.1`. El ejemplo usa el puerto 27018 y un directorio nuevo dentro de la copia. Sustituya la ruta del ejecutable por la que corresponda a su instalación.

```
New-Item -ItemType Directory -Force .\local-auth-data | Out-Null
& '<ruta instalada>\mongod.exe' --dbpath '.\local-auth-data' --bind_ip 127.0.0.1 --port 27018
```

Terminal 2 · Servidor

Desde la raíz de la copia, prepare el entorno y defina los valores de la prueba.

```
python -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
$env:MONGO_URI = 'mongodb://127.0.0.1:27018'
$env:MONGO_DB_NAME = 'sec_copia_login_local'
$env:SEC_PRIMARY_ADMIN_EMAIL = 'admin@prueba.local'
$env:SECRET_KEY = .\.venv\Scripts\python.exe `
  -c 'import secrets; print(secrets.token_urlsafe(48))'
$env:APP_PUBLIC_URL = 'http://127.0.0.1:3014'
$env:ALLOWED_ORIGINS = 'http://127.0.0.1:3014'
$env:SEC_TEMPLATE_READY = 'true'
.\.venv\Scripts\python.exe scripts\bootstrap_admin.py
.\.venv\Scripts\python.exe -m uvicorn main:app --host 127.0.0.1 --port 8014
```

El script solicita una contraseña de al menos 12 caracteres y guarda su hash. Se ejecuta una vez con la base sin usuarios. Si se interrumpe el proceso, compruebe el resultado antes de reintentarlo. Para conservar las sesiones, mantenga la misma clave en reinicios mediante el gestor de secretos del entorno.

Abra `http://127.0.0.1:8014/health`. Un resultado saludable confirma la respuesta de la API y su conexión con MongoDB. Continúe en la misma terminal para conservar las variables del proceso.

05. Web y verificación de sesión

Terminal 3 · Aplicación web

Abra otra terminal. Mantenga abiertos MongoDB y la API.

```
Set-Location '<ruta de SEC-COPIA-BASE>\panic-app'
npm.cmd ci
$env:PORT = '3014'
$env:HOST = '127.0.0.1'
$env:BROWSER = 'none'
$env:REACT_APP_SEC_PREVIEW_ONLY = 'false'
$env:REACT_APP_SEC_AUTH_ONLY = 'false'
$env:REACT_APP_SEC_LOCAL_PREVIEW = 'false'
$env:REACT_APP_SEC_TEMPLATE_READY = 'true'
$env:REACT_APP_SEC_MUNICIPALITY_NAME = 'Municipio de prueba'
$env:REACT_APP_SEC_API_URL = 'http://127.0.0.1:8014'
npm.cmd start
```

Criterios de la prueba

1. Abra `http://127.0.0.1:3014` e ingrese con el correo definido en `SEC_PRIMARY_ADMIN_EMAIL` y la contraseña que eligió.
2. Compruebe que se abra el panel administrativo. El navegador debe comunicarse con la API del puerto 8014.
3. Cierre sesión y vuelva a entrar. El servidor recibe `POST /login`, permite verificar la sesión con `GET /auth/session` y procesa la salida con `POST /auth/logout`.
4. Verifique también una credencial incorrecta y una sesión cerrada con una cuenta de prueba. Mantenga los intentos dentro de los límites configurados.

Origen y puerto

Para CORS, `http://localhost:3014` y `http://127.0.0.1:3014` son orígenes distintos. Use la dirección del ejemplo o ajuste expresamente la lista permitida. Reinicie la API al cambiar variables del servidor y reinicie o recompile la web al cambiar `REACT_APP_*`.

06. Usuarios, roles y sesión

Alta de cuentas

- **Primer administrador.** Se crea desde el servidor con `scripts/bootstrap_admin.py`. El registro queda con `rol: admin` y estado activo.
- **Operador.** El administrador usa Usuarios. La ruta documentada es `POST /usuarios/operadores`. Revise la política de IP autorizada y pruebe desde el puesto real.
- **Fiscal.** El administrador usa Usuarios. La ruta documentada es `POST /usuarios/fiscales`. Compruebe los permisos del mapa y de la gestión de oficios.
- **Ciudadano.** La persona usa Crear cuenta cuando el municipio habilita `REACT_APP_SEC_CITIZEN_REGISTRATION_ENABLED`. `POST /registro/usuario` crea el perfil ciudadano en estado pendiente. La administración lo aprueba según su procedimiento.
- **Otros perfiles.** Valide los procedimientos habilitados para dirección, patrullaje y patentes con el administrador municipal y compruebe cada interfaz antes del piloto.

Identidad institucional alternativa

Si el municipio adopta un proveedor institucional, revise `main.py` y `AUTH_INTEGRATION.md`. El servidor verifica la identidad, asigna el rol institucional y emite la sesión S.E.C. En OIDC, valide emisor, destinatario, firma y vigencia. Autorice al administrador mediante una asignación institucional verificable.

Pruebas de permisos

Use una cuenta por perfil. Verifique las operaciones permitidas y que las operaciones fuera de su rol se rechacen también en la API. Compruebe expiración, cierre, recuperación y segundo factor cuando corresponda. `DEFAULT_OPERATOR_AUTHORIZED_IP=127.0.0.1` sirve como referencia local.

07. Datos operativos y archivos

Datos

Revise `dataServiceConnections.js`, `dataStorePrimitives.js` y `operationalDataApi.js` bajo `panic-app/src`. El adaptador base devuelve lecturas vacías y rechaza escrituras. Implemente el contrato que espera cada pantalla.

1. Identifique las colecciones y campos que necesita el flujo. Defina identificadores, estados y permisos.
2. Conecte lecturas, escrituras y suscripciones que el flujo requiere. Valide las autorizaciones en la API o en el servicio correspondiente.
3. Verifique persistencia desde otra sesión y después de reiniciar el servicio. Simule caída de conexión y compruebe que la interfaz informa el fallo.
4. Mida tiempos de consulta y volumen con datos ficticios representativos. Compruebe el comportamiento cuando llegan cambios concurrentes.

Si se cambia MongoDB por otro motor para autenticación, adapte las consultas, índices y operaciones de `main.py`. Después vuelva a probar el alta inicial, la sesión, los permisos y la recuperación.

Evidencias y adjuntos

Revise `panic-app/src/utils/storageUploads.js`. Defina dónde se guardan imágenes, documentos y audio, quién puede leerlos y qué tamaños se admiten. Vincule cada archivo con su registro y pruebe subida, consulta, rechazo por permisos, expiración y recuperación desde respaldo. Incluya en la prueba un adjunto interrumpido y un archivo que exceda el tamaño admitido.

08. Cartografía, cámaras e identidad

Vista territorial

Configure `REACT_APP_SEC_MAP_LAT`, `REACT_APP_SEC_MAP_LNG` y `REACT_APP_SEC_MAP_ZOOM` con el encuadre municipal. Revise `panic-app/src/Mapa.js` y `panic-app/src/components/maps/basemapSources.js`. Acredite que la fuente cartográfica y sus atribuciones corresponden al uso previsto.

Inventario

Para cada punto, prepare un identificador, nombre, sector, coordenadas y estado. Defina quién actualiza el inventario y con qué frecuencia. Verifique ubicación y visibilidad con una muestra y compruebe que el contador de puntos se distingue del total de cámaras instaladas.

Pruebe encuadre inicial, capas, zoom, filtros y ficha de cámara por cada rol autorizado. Compruebe qué estado muestra la interfaz cuando el servicio está caído o la información es antigua. Si se incorpora video, valide su flujo y permisos por separado.

Marca y canales públicos

Complete nombre, logo, teléfonos y enlaces oficiales. Revise `REACT_APP_SEC_MUNICIPALITY_NAME`, `REACT_APP_SEC_EMERGENCY_PHONE`, `REACT_APP_SEC_TERMS_URL` y `REACT_APP_SEC_PRIVACY_URL`. Use contenido aprobado por el municipio y pruebe su presentación en escritorio y móvil.

09. Microsoft para planillas

Esta integración es opcional y se autoriza al usar las planillas. La autenticación del panel S.E.C. utiliza su propia sesión.

1. Registre una aplicación propia del municipio en su directorio Microsoft. Configure el tipo de aplicación pública de una sola página que requiere el conector.
2. Registre la URI de retorno exacta usada por la web. Para el ejemplo local, use `http://127.0.0.1:3014`. Registre la URL HTTPS correspondiente para producción.
3. Revise los permisos delegados del conector. La guía de esta base documenta `User.Read` y `Files.ReadWrite`. Obtenga el consentimiento institucional correspondiente.
4. Defina `REACT_APP_SEC_MICROSOFT_CLIENT_ID`, `REACT_APP_SEC_MICROSOFT_TENANT_ID` y `REACT_APP_SEC_MICROSOFT_ACCOUNT`. Compruebe que los tres valores correspondan a la misma institución. Reinicie o recompile la web.
5. Entre a S.E.C., abra el módulo de planillas y autorice con la cuenta prevista. Compruebe lectura y escritura sobre un archivo de prueba.

Si Microsoft rechaza la autorización

Revise primero los tres valores públicos. Si el mensaje menciona `client_id`, confirme que la compilación recibió el ID de aplicación. Si menciona la URI de retorno, compare protocolo, host, puerto y ruta del valor registrado con la solicitud. Si el problema aparece al acceder a un archivo, revise el consentimiento, la cuenta y el permiso sobre ese archivo.

El punto principal del cliente es `panic-app/src/utils/institutionalAccountAuth.js`. Los módulos `*Workbook.js` gestionan las planillas correspondientes.

10. Comunicaciones y alertas

Qué debe configurar el municipio

- **Correo.** Revise `SMTP_*` en la plantilla del servidor. Pruebe remitente, destinatario, entrega y comportamiento cuando falle el proveedor.
- **Web Push.** Revise `WEB_PUSH_*`, el registro del navegador y su permiso de notificaciones. Compruebe entrega en navegadores y dispositivos del piloto.
- **SMS y avisos móviles.** Revise `enviar_sms_recuperacion`, `send_mobile_poster_push` y `send_mobile_remote_panic_push` en `main.py`. Son puntos de integración de la base.
- **Videollamadas.** Revise `VideoCallPanel.js`, `/video/token` y `/webrtc/ice-servers`. Requiere un servicio y una implementación coherente entre web y API.

Prueba de una alerta

1. Coordine una ventana de prueba con la central e identifique los registros como simulados.
2. Active la prueba desde el dispositivo acordado y registre la hora de envío, recepción en la central y confirmación de atención.
3. Repita con las redes y equipos que se utilizarán. Compare el resultado con el objetivo de entrega acordado.
4. Compruebe qué sucede con la aplicación en segundo plano, cuando se pierde conectividad y cuando se recupera. Aplique el canal de contingencia definido por el municipio.

La llegada de una notificación y el registro de atención son verificaciones separadas. Documente ambas antes de habilitar un flujo de emergencia.

11. Producción, aceptación y continuidad

Puesta en marcha

1. Registre la versión aprobada y configure los dominios HTTPS de web y API. Restrinja CORS a los orígenes correspondientes.
2. Cargue configuración privada en el servidor. Compile en `panic-app` con `npm.cmd ci` y `npm.cmd run build`. Compruebe que las opciones de vista previa estén desactivadas.
3. Publique primero en el entorno de prueba. Verifique `/health`, login, sesión, guardado, archivos y cada integración que se vaya a activar.
4. Asegure la disponibilidad de la versión anterior y confirme el procedimiento de reversión, incluidas las migraciones de datos.
5. Tras desplegar, repita las verificaciones desde la URL final. Revise registros de errores, certificados y alertas de disponibilidad.

Evidencias mínimas de aceptación

- **Acceso.** Cuenta por rol, resultado esperado, permisos en API y prueba de cierre y expiración.
- **Datos.** Identificador ficticio de un registro que se guardó y reapareció en otra sesión.
- **Archivos.** Evidencia de subida y lectura autorizada, y rechazo cuando faltan permisos.
- **Servicios opcionales.** Resultado de entrega, contingencia y responsable operativo por cada servicio habilitado.
- **Recuperación.** Fecha y resultado de una restauración de base y archivos en un entorno aislado.

Si falla la puesta en marcha

Detenga la ampliación del piloto, conserve los registros necesarios para diagnosticar y avise a la contraparte. Si corresponde, aplique el procedimiento de reversión acordado. Cuando cambie el esquema de datos, compruebe la compatibilidad antes de volver a una versión anterior de la aplicación.

12. Diagnóstico rápido

La API no abre o `/health` falla

Compruebe que Uvicorn sigue abierto, que escucha en 8014 y que MongoDB sigue escuchando en 27018 para el ejemplo. Revise el error de la terminal de la API y confirme que las variables pertenecen a esa sesión de PowerShell.

La web muestra «Failed to fetch» pero `/health` responde

Compare la URL exacta de la web con `ALLOWED_ORIGINS` y revise a qué URL envía la solicitud `REACT_APP_SEC_API_URL`. Abra la pestaña Red del navegador para ver si falla la conexión, la solicitud previa de CORS o la respuesta HTTP. Reinicie el proceso afectado después de corregir su configuración.

El alta inicial informa que ya existen usuarios

Verifique el nombre de la base y el puerto. El script protege la inicialización cuando ya hay cuentas. Si es una instalación en uso, aplique su procedimiento de recuperación de administrador.

Entré, pero el tablero muestra cero registros

Confirme si el adaptador operativo está conectado y si el rol tiene acceso a los datos. Un acceso correcto prueba la sesión. Los datos del tablero requieren además su conector y una prueba de persistencia.

El navegador lleva a Microsoft o la planilla no abre

Distinga si ocurrió al ingresar al sistema o al abrir una planilla. Revise el modo local y las variables del cliente. Si falla solo la planilla, siga la sección 09.

Un operador entra localmente, pero desde su puesto falla

Confirme el estado y el rol de la cuenta, la conectividad desde el puesto y las IP autorizadas. Revise cómo el despliegue identifica la IP del cliente cuando hay un proxy.

La vista quedó en blanco o sigue apuntando al puerto anterior

Revise el primer error de compilación o de consola, confirme el puerto y reinicie la web después de cambiar su entorno. Identifique si la pestaña aún muestra una compilación anterior.

13. Consulta técnica y referencias

Qué incluir al solicitar ayuda

- Versión o commit de la base y sistema operativo.
- Entorno y etapa: vista previa, acceso real, piloto o producción.
- Módulo, rol, paso ejecutado y resultado esperado.
- Texto exacto del error, hora y código HTTP si aparece. Use registros depurados y valores de ejemplo. Omita contraseñas, claves, tokens y datos personales.
- Resultado de `/health` cuando el problema sea de conexión.

Cómo usar la respuesta del asistente

Busque una respuesta referida a la misma versión y valide primero los pasos en prueba. Si el problema depende de un proveedor, de permisos institucionales o de datos que el manual no cubre, derive el caso al responsable municipal con los antecedentes indicados arriba.

Documentación del repositorio

- `README.md` describe el alcance y el arranque rápido.
- `PRIMER_LOGIN_LOCAL.md` documenta la prueba con base aislada, API y web.
- `AUTH_INTEGRATION.md` describe roles, rutas y la integración de identidad.
- `INTEGRACION_MUNICIPAL.md` ubica los conectores y explica la conexión de planillas.
- `GUIA_IMPLEMENTACION_MUNICIPAL.md` reúne requisitos, pruebas y criterios de entrega.
- `.env.example` y `panic-app/.env.example` enumeran los parámetros de servidor y web.